



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT ON

Online Shopping Cart Simulation

Subject Name C Programing

SUBJECT CODE 25CAH-101

SUBMITTED BY:

Name : Kanak Rana

UID : 25BCD10104

Branch :BCA'Data science'

Group: B

Section: 25BCD'2B'

Semester : 1

SUBMITTED TO:

Name : Dharam Pal Singh

D.O.SUBMITION : _____

Designation: Assistant Proffeser

Sign:_____

Project Report: Online Shopping Cart Simulation

1. Title

Online Shopping Cart Simulation

This is a console-based C programming project simulating a basic e-commerce shopping cart system, demonstrating how users can add, view, and remove products just like on any online store.

2. Aim / Introduction of the Project

The **aim** of this project is to design and implement a simplified model of an online shopping cart using the C programming language. Online shopping carts are a critical component of modern e-commerce platforms, powering the process where customers select products, review their selections, and proceed to checkout.

Introduction:

This project replicates the main features of an online retailer's cart system: presenting a fixed product catalog, enabling product selection, maintaining a shopping cart, and calculating final totals. It serves as both a learning tool and a functional demonstration of real-world programming concepts such as arrays, structures, functions, and loops, all within a user-friendly terminal interface.

3. Objective

- **To simulate an online shopping experience** within a console environment.
- **To implement all core functionalities** such as listing products, adding/removing items, checking the current cart, and finalising purchases.
- **To give students practical experience** in structured programming, modular code development, and user-driven logic handling.

- **To illustrate the data flow and logic** of basic e-commerce cart systems relevant for more advanced commercial and academic pursuits.

4. Overview of E-Commerce Concept Used

E-commerce (Electronic Commerce) is the buying and selling of goods or services using the internet.

This project mimics:

- **Product Catalog Browsing:** Users can view a predefined selection of products (names and prices).
- **Cart Management:** Users may add selected products to a virtual cart with specific quantities.
- **Order Modification:** Products can be removed before purchase, and quantities adjusted.
- **Checkout Calculation:** The system computes the total cost for all items in the cart, preparing the user for payment (though real payment is not implemented).

This simulation mirrors the essential path taken by a shopper on actual e-commerce sites—explore, select, adjust choices, and checkout.

5. Working Model / Process Flow

The shopping cart simulation follows a structured workflow:

Step-by-step Process:

1. **Main Menu Display:** User is repeatedly shown a list of options.
 - View available products
 - Add item(s) to the cart
 - View current cart
 - Remove item(s) from the cart

- Checkout & Exit

2. **Product Representation:** Each product is stored within a structure containing its name and price. The available product list is stored in an array.

3. **Cart Handling:**

- When a user adds a product, it is stored in a “cart” array along with the chosen quantity.
- When the user wishes to delete an item, it is removed from the cart array.
- When viewing the cart, a summary table (item, price, quantity, total for item, overall total) is displayed.

4. **Loops and Logic:**

- The program uses `while` and `for` loops to handle repeated actions and array traversals.
- Input validation ensures the user cannot perform invalid operations (e.g., adding nonexistent products).

5. **Exit:**

- On choosing checkout, the application displays the order summary and thanks the user before closing.

Visual process charts or diagrams can further illustrate these stages.

6. Benefits and Limitations

Benefits:

- *Educational Value:* Provides hands-on practice with structures, arrays, and control flow.
- *Real-World Relevance:* Models a standard business function found in almost all e-commerce portals.

- *Modularity*: Code is broken into functions making it easy to understand, debug, and extend.
- *User Experience*: Simulates menu-based choices, mirroring typical online interactions.

Limitations:

- *Static Product List*: Product catalog is hard-coded and does not allow dynamic updating or database connections.
- *No Persistent Storage*: Cart and product data only exist as long as the program runs—upon closing, all data is lost.
- *No Authentication or User Profiles*: Does not simulate accounts, logins, or personalised experiences.
- *No Real Payment Processing*: Checkout only summarises totals, with no transaction or order management features.
- *Not Scalable*: Intended for demonstration and small-scale use only, not suitable for a real business environment without major enhancements.

7. Technology Technology:

- *Programming Language*: C (chosen for its widespread use and educational focus for learning procedural programming)
- *Data Structures*: Arrays (for product and cart storage), Structures (for defining products and cart items)
- *Tools/IDE*: Compatible with Turbo C, Dev-C++, GCC, Code::Blocks, or any standard C IDE

8. Risk and Security Measures

For this simulation, risks and security are minimal since no real funds, sensitive user credentials, or data transmissions are involved.

However, in real-world scenarios:

- Risks include unauthorised access, transaction fraud, data breaches, and exploitation of software vulnerabilities.
- Security measures could include HTTPS encryption, server-side validation, PCI compliance, password hashing, token-based authentication, input sanitisation, and rate limiting.
- Regular updates and security audits would also be required.

Code overview

Here's an **overview of the main components and logic in the provided Online Shopping Cart C code**. This explanation will help you understand each part and how they work together:

1. Product Structure and List

- **Structure Definition:**

C

```
typedef struct {  
    char name[30];  
    float price;  
} Product;  
•
```

Each product has a name and price, grouped together in a structure for easy handling.

- **Product Array:**

C

```
Product products[MAX_PRODUCTS] = {
```

- {"Helmet", 1500.0},
- {"Gloves", 500.0},
- {"Airbag Vest", 8500.0},
- {"Knee Guards", 900.0},
- {"Rain Jacket", 1800.0}
- };
-

A static array lists all the products available for purchase. You can easily display these to the user or fetch their details using an index.

2. Cart Structure and List

- **Cart Item Structure:**

C

```
typedef struct {
```

- Product product;
- int quantity;
- } CartItem;
-

Links a product to a quantity, indicating how many units of a particular product the user wants.

- **Cart Array:**

C

```
CartItem cart[MAX_CART_SIZE];
```

- `int cartSize = 0;`
-

3. Functions

- **displayProducts():**
Shows all available products, printing ID, name, and price for each. Helps users decide what to buy.
- **addToCart():**
Lets the user choose a product (by ID) and a quantity to add to their cart. If the product was already added, the function increases the quantity.
- **displayCart():**
Displays current cart items, their prices, quantities, and individual as well as total costs. Useful for reviewing before checkout.
- **removeFromCart():**
Allows the user to specify which cart item to remove. It then updates the cart array to contain only the remaining items.

4. Main Menu and Workflow

- **main() Function:**
Runs the interactive loop, presenting the user with options (view products, add, view cart, remove, checkout/exit). According to user input, the corresponding function is called until the user decides to exit.

5. Input Validation

- When a user chooses a product or a cart item (by ID), checks ensure that only valid IDs are accepted.
- Cart array management prevents more items than allowed (`MAX_CART_SIZE`).

6. Modular Design

- Breaking the code into *functions* for each feature keeps logic separated and code readable.
- Structures (`Product`, `CartItem`) organise related data, making manipulation straightforward.

9. Findings / Analysis

- **Feasibility:** Creating a shopping cart is feasible with fundamental programming skills.
- **Code Structure:** Modular design (dividing into logical functions, such as `addToCart`, `displayCart`) improves maintainability and clarity.
- **User Input Processing:** Ensuring user inputs are correctly validated is crucial to preventing program crashes and unexpected behaviours.
- **Learning Outcome:** The project helps solidify the link between theoretical data structures and practical business logic, showing how arrays/structures map to real operations like a shopping cart.

10. Conclusion

This project successfully replicates the core shopping cart mechanism employed in most digital commerce platforms.

Achievements:

- User can interactively view, add, and remove products.
- Accurate total calculation.

- Demonstrates important academic concepts: modularity, structures, arrays, loops, and menu-driven programs.

It lays the groundwork for more complex systems involving databases, file handling, graphical interfaces, and real payments, and is an excellent practical step for beginners and intermediate programmers.

11. Learning Outcomes

- *Practical Programming:* Developed confidence in writing and breaking down real projects in C using modular functions.
- *Theoretical Concepts in Action:* Saw direct application of data structures (arrays, structures) and control logic.
- *Understanding System Design:* Gained insight into how online systems organize workflows.
- *Preparation for Future Topics:* Experience with user menus, validation, and data management prepares for higher-level concepts and more advanced development tools.

12. References

- *GeeksforGeeks:* “Shopping Cart Project Using C Language”
- *Online Documentation and forums* (Stack Overflow, TutorialsPoint)
- *Project Template Document* (your attached case study report format)

13. Screenshots

Insert relevant screenshots of:

```
--- Online Shopping Cart ---  
1. View Products  
2. Add to Cart  
3. View Cart  
4. Remove from Cart  
5. Checkout & Exit
```

- Output displaying the main menu
- Product list

```
Available Products:
ID      Name      Price
1       Helmet     1500.00
2       Gloves     500.00
3       Airbag Vest 8500.00
4       Knee Guards 900.00
5       Rain Jacket 1800.00
```

- Cart summary and checkout

```
Your Cart:
ID      Name      Price  Qty  Total
1       Gloves     500.00  40  20000.00
Cart Total: 20000.00
```

Teacher Signature: _____